Exam 2: Exploring Data – Version B

Exam Agreement

This exam is an **individual assessment** and you should neither offer help to nor receive help from another person.

Cell phones are to be handed in to the instructor and all other electronic devices must be stored in bags at the front of the room.

No smart glasses or other wearable electronic devices are to be powered on during this exam except with accommodations.

You may use **no physical or electronic** resource to complete the work.

By signing, you agree that you have neither given nor received unauthorized aid on this examination.

Printed Name: _____

Chapman ID: _____

Signed: _____

Section: _____ **Date:** _____

Include this signed page as the first page of your submitted work.

Exam Instructions

You must answer every question on the exam to the best of your ability. Include all appropriate code, syntax, and functions that would lead to the code to run successfully.

Question 1: Loading Data

Your boss has given you an Excel file that has results from a series of experiments another team member has conducted on a new anti-viral drug candidate. You've converted the Excel file to a CSV file. Write the code to load the data set into R as a data frame named `plaque_results`.

Question 2: Cleaning Data

Your data frame from Question 1 (`plaque_results`) has four columns: the date of the trial (`Date`), the trial number (`Trial`), the number of plaques in the control condition (`Control`) and the number of plaques in the experimental condition (`Experimental`). You'd like to plot these data using `ggplot2`, but they need to be in the long format. Pivot the data into the long format and store it in a new object called `plaque_long`.

Question 3: Basic Data Analysis

The cleaned data from Question 2 involve trials conducted as plaque assays, where the drug is introduced into a culture with cell media and virus, and the results are counts of plaques (or areas where virus has killed an area of the cells in culture). Experiments were run with controls (an inactive form of the drug) and experimental (with the active drug). Plot the data using `ggplot2` so that you can visually assess whether the drug was successful and reducing the number of viral plaques. What geometric object are you choosing and why?

Question 4: Troubleshooting Compound Code

The following code produces an error:

```
robo.data <- read.csv("robo-mussel.csv")
robo_clean <- robo.data %>%
  clean_names() %>%
  remove_empty(c("rows", "cols"))
```

The error is:

```
Error in robo.data %>% clean_names() %>% remove_empty(c("rows", "cols")) :
  could not find function "%>%"
```

What is the problem and how do you fix it?

Question 5: Plotting with Base Graphics

You have a subset of the `CO2` data set named `CO2_Quebec` (pictured in Fig. 1). Using base graphics, create a scatterplot of uptake versus concentration with the points colored by Plant. Add lines for each plant that are the same colors as the points. Include a legend.

	Plant	Type	Treatment	conc	uptake
1	Qn1	Quebec	nonchilled	95	16.0
2	Qn1	Quebec	nonchilled	175	30.4
3	Qn1	Quebec	nonchilled	250	34.8
4	Qn1	Quebec	nonchilled	350	37.2
5	Qn1	Quebec	nonchilled	500	35.3
6	Qn1	Quebec	nonchilled	675	39.2
7	Qn1	Quebec	nonchilled	1000	39.7
8	Qn2	Quebec	nonchilled	95	13.6
9	Qn2	Quebec	nonchilled	175	27.3
10	Qn2	Quebec	nonchilled	250	37.1
11	Qn2	Quebec	nonchilled	350	41.8
12	Qn2	Quebec	nonchilled	500	40.6
13	Qn2	Quebec	nonchilled	675	41.4
14	Qn2	Quebec	nonchilled	1000	44.3
15	Qn3	Quebec	nonchilled	95	16.2
16	Qn3	Quebec	nonchilled	175	32.4
17	Qn3	Quebec	nonchilled	250	40.3
18	Qn3	Quebec	nonchilled	350	42.1
19	Qn3	Quebec	nonchilled	500	42.9
20	Qn3	Quebec	nonchilled	675	43.9
21	Qn3	Quebec	nonchilled	1000	45.5

Figure 1: The CO2_Quebec data set.

Question 6: Plotting with ggplot2

Recreate the same plot as in Question 5 using `ggplot2`.

Question 7: How We See

Describe the effect that self inhibition of retinal ganglion cells have in our perception of light.

Question 8: Visual Processing and Attention

Name two "fast-processing" differences and describe an example of how they can be used in a plot to encode data.

Question 9: Providing Constructive Feedback

Provide constructive feedback for the visualization of air versus water temperature of a variety of sampling sites in Fig. 2.

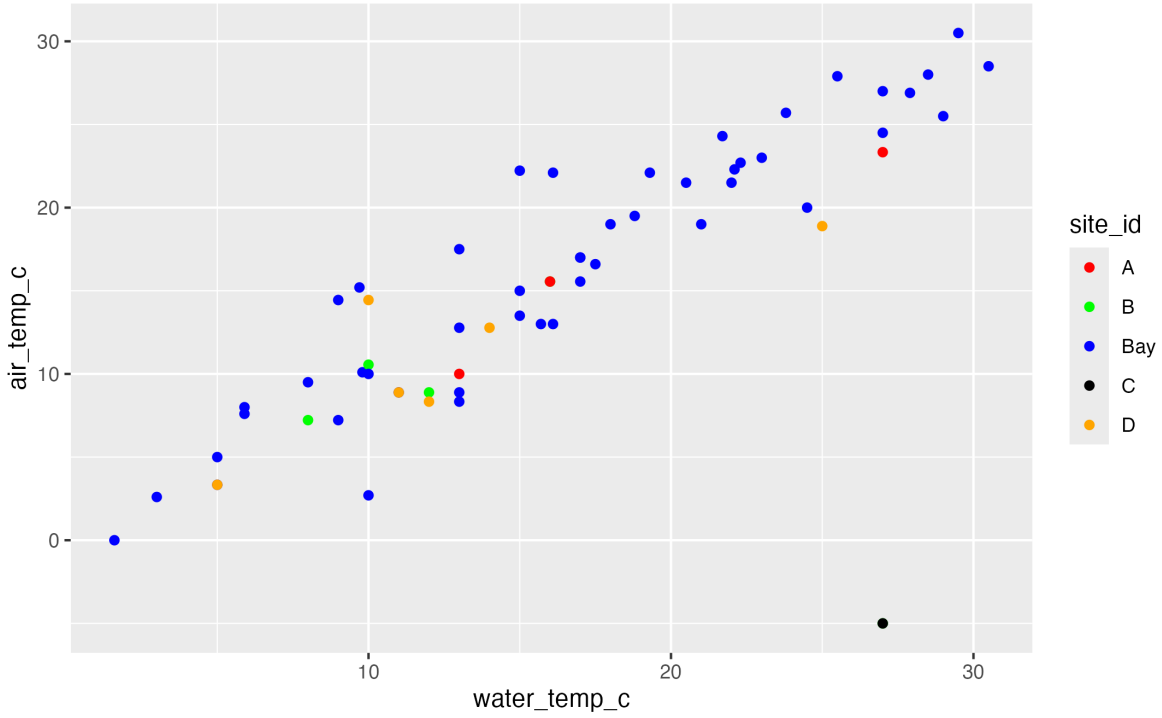


Figure 2: Air versus water temperatures at five sampling sites

Question 10: Ethics of Data Visualization

Describe the ethical obligation you have when visualizing data. Why is this important?

List of Functions for Exam 2

- `+, -, *, /, ^` – arithmetic operators
- `sqrt()` – square root
- `log()` – logarithm
- `seq()` – sequence creation
- `rep()` – value/vector repetition
- `sort()` – vector sort
- `length()` – determine vector length
- `[ ]` – vector subset/extraction, list slicing, multiple members
- `sum()` – sum all vector elements
- `prod()` – multiply all vector elements
- `mean()` – averages all vector elements
- `sd()` – takes standard deviation of all vector elements
- `matrix()` – create a matrix
- `rbind()` – create a matrix (bind rows)
- `cbind()` – create a matrix (bind columns)
- `dim()` – get matrix dimensions
- `nrow()` – get number of rows
- `ncol()` – get number of columns
- `[ , ]` – matrix/array/data frame subsetting
- `diag()` – diagonal elements/identity matrix
- `array()` – create an array
- `list()` – create a list
- `[[ ]]` – unnamed member reference list
- `$` – get named member/variable
- `data.frame()` – create data frames
- `==, !=, >, <, >=, <=` – relational operators
- `any()` – checks whether any entries are true
- `all()` – checks whether all entries are true
- `&&, &, ||, |, !` – logical operators
- `which()` – determines indices of `TRUE`s
- `nchar()` – gets the number of characters in a string
- `cat()` – concatenates strings (no return)
- `paste()` – pastes strings (returns a string)
- `\` – escape character
- `substr()` – Subsets a string
- `sub()`, `gsub()` – string matching and replacement
- `factor()` – creates a factor vector
- `levels()` – gets levels of a factor
- `cut()` – creates factor from continuum
- `is.infinite()` – element-wise check for `Inf`
- `is.finite()` – element-wise check for finiteness
- `NaN` – value for invalid numerics
- `is.nan()` – element-wise check for `NaN`
- `NA` – value for missing observation
- `is.na()` – element-wise check for `NA` or `NaN`
- `na.omit()` – delete all `NA`s and `NaN`s
- `NULL` – value for empty
- `is.null()` – check for `NULL`
- `attributes()` – list explicit attributes
- `attr()` – obtain specific attribute
- `dimnames()` – get array dimension names
- `class()` – get object class
- `is._()` – object-checking functions
- `as._()` – object-coercion functions
- `unlist()` – object-coercion from list to vector
- `runif()` – draws pseudo-random numbers from a uniform distribution
- `sample()` – draws numbers from a sample given by user

- `plot()` – makes a plot in base graphics
- `~` – creates formula object
- `points()` – adds points to a plot in base graphics
- `lines()` – adds points to a plot in base graphics
- `legend()` – adds a legend to a plot in base graphics
- `pivot_wider()` – restructure data to wide format (tidyr)
- `pivot_longer()` – restructure data to long format (tidyr)
- `ggplot()` – initiates a plot using ggplot2
- `geom_x()` – geometric object layers in ggplot2
- `ggtitle()` – title a ggplot
- `labs(), xlab(), ylab()` – alter axis labels in ggplot2
- `lims(), xlim(), ylim()` – alter axis limits in ggplot2
- `summary()` – produces summary stats on a data frame
- `head()` – prints first lines of a data frame
- `tail()` – prints last lines of a data frame
- `IQR()` – computes interquartile ranges
- `median()` – computes median
- `table()` – builds a contingency table of counts
- `quantile()` – produces sample quantiles
- `tapply()` – apply function to groupings in data set
- `apply()` – apply function to row or column of data set
- `read.table()` – imports generic text file
- `read.csv()` – imports comma separated value file
- `clean_names()` – standardizes column names in data frame (janitor)
- `remove_empty()` – removes empty columns and/or rows in data frame (janitor)
- `%>%` – tidy pipe operator (tidyr/janitor)
- `write.csv()` – exports object to text csv file